Relatório de listas de prioridades implementadas como heaps binários

alunos: Guilherme Thomasi Ribas e Pedro Henrique Magalhães

Definição

- Heaps: Um heap binário é uma estrutura de dados baseada em uma árvore binária completa que satisfaz uma propriedade de ordenação entre seus elementos. É utilizado principalmente na implementação de filas de prioridade, permitindo inserir elementos e remover o elemento de maior ou menor prioridade de maneira eficiente. Por ser uma árvore binária completa, todos os níveis são preenchidos, exceto possivelmente o último, que é preenchido da esquerda para a direita. Os heaps binários podem ser representados por vetores, dispensando a necessidade de utilizar ponteiros para representar os nós da árvore.

- Heap de mínimo: Um heap de mínimo é uma estrutura na qual o valor de cada nó pai é menor ou igual aos valores de seus filhos. Dessa forma, o menor elemento da estrutura sempre se encontra na raiz da árvore.

- Heap de máximo: Um heap de máximo é uma estrutura na qual o valor de cada nó pai é maior ou igual aos valores de seus filhos.Consequentemente, o maior elemento da estrutura sempre se encontra na raiz da árvore.

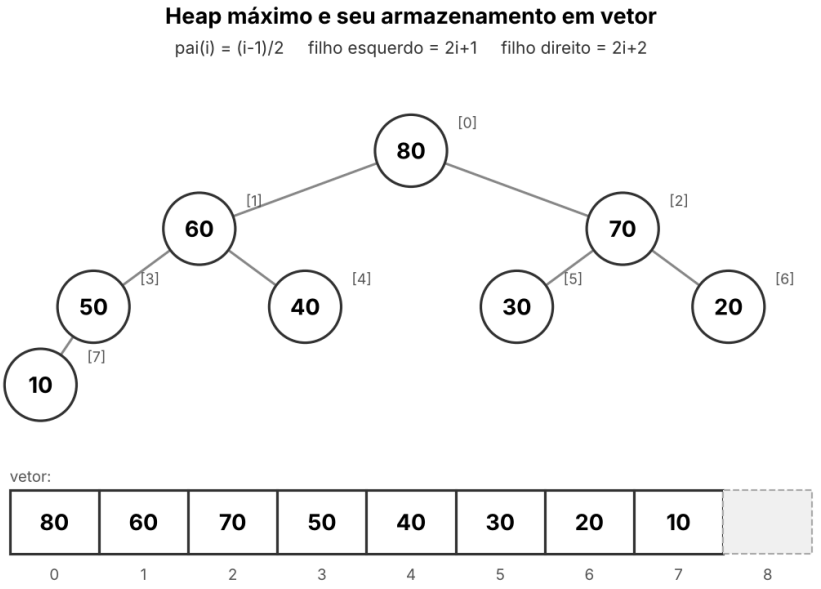
Propriedades

- Árvore binária completa: todos os níveis, exceto possivelmente o último, estão completamente preenchidos. O último nível é preenchido da esquerda para a direita.
- Propriedade de heap: no heap de mínimo, o pai é menor ou igual aos filhos; no heap de máximo, o pai é maior ou igual aos filhos.
- Elemento prioritário na raiz: o menor elemento está na raiz do min-heap, enquanto o maior está na raiz do max-heap.
- Representação em vetor: os elementos podem ser armazenados em um vetor, utilizando índices para acessar os nós pais e filhos.
- Altura logarítmica: devido à sua estrutura de árvore completa, a altura de um heap binário é $O(\log n)$, sendo n o número de elementos.
- Não é totalmente ordenado: um heap garante a relação de prioridade entre pais e filhos, mas não estabelece uma ordenação completa entre todos os elementos.

Armazenamento em vetor

Como a árvore é completa, ela pode ser guardada em um vetor sem ponteiros e sem espaços vazios no meio: os nós são colocados em ordem de nível (varredura por níveis), da esquerda para a direita. Com índices começando em 0, as relações entre os nós são calculadas por aritmética simples:

Relação	Índice	Condição
Raiz	0	n > 0
Pai de i	(i − 1) / 2 (divisão inteira)	i > 0
Filho esquerdo de i	2i + 1	2i + 1 < n
Filho direito de i	2i + 2	2i + 2 < n



Uma estrutura em C que representa o heap pode ser:

```
typedef struct {
    int *dados;   /* vetor que guarda o heap       */
    int  n;       /* quantidade de elementos       */
    int  cap;     /* capacidade do vetor           */
} Heap;
```

A vantagem dessa representação é o uso compacto da memória e o acesso direto ao pai e aos filhos de qualquer nó. O elemento de maior prioridade é consultado em tempo constante, em dados[0].

- Inserção

A inserção deve preservar as duas propriedades. Para manter a árvore completa, o novo elemento é colocado na primeira posição livre do vetor(n). Isso pode violar a propriedade de heap, pois o novo valor pode ser maior que o de seu pai. Para corrigir, executa-se o **upheap,** enquanto o elemento for maior que seu pai, os dois trocam de lugar.

Algoritmo:

1. Se n = capacidade, o heap está cheio (erro ou redimensionamento).
2. Colocar o novo elemento em dados[n].
3. Executar upheap a partir do índice i = n.
4. Incrementar n.

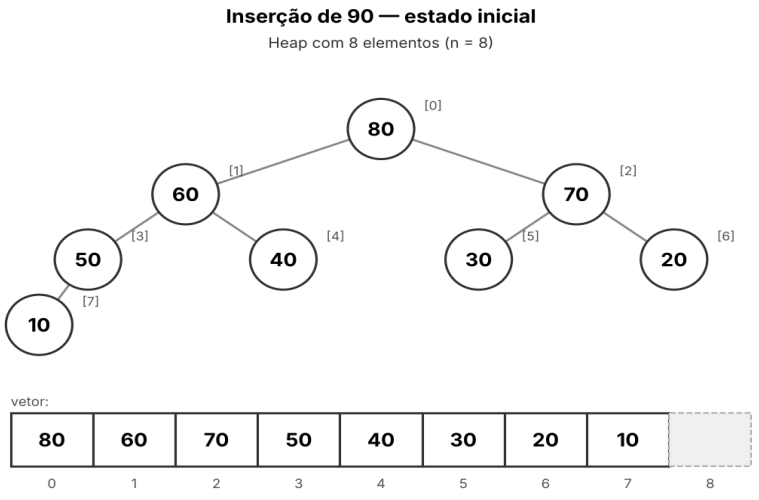
```
void upheap(Heap *h, int i) {
    while (i > 0) {
        int pai = (i - 1) / 2;
        if (h->dados[i] <= h->dados[pai])
            break;
        int tmp = h->dados[i];
        h->dados[i] = h->dados[pai];
        h->dados[pai] = tmp;
        i = pai;
    }
}

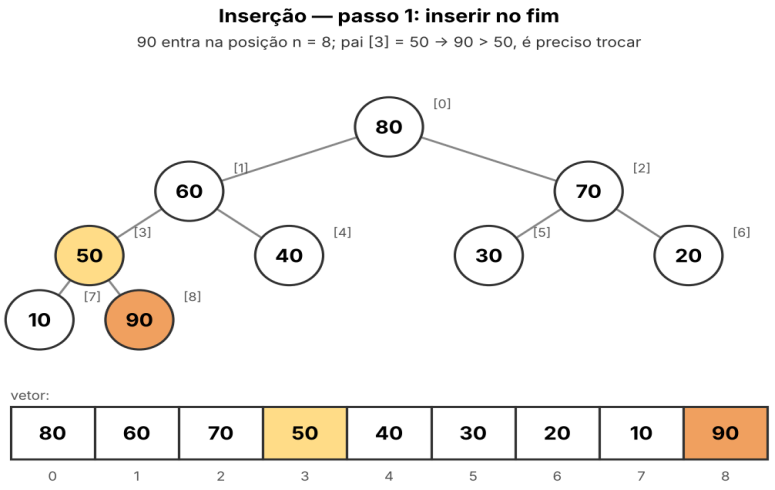
int inserir(Heap *h, int x) {
    if (h->n == h->cap) return 0;       /* heap cheio          */
    h->dados[h->n] = x;                 /* última posição      */
    upheap(h, h->n);
    h->n++;
    return 1;
}
```

Como cada troca sobe um nível e a altura de uma árvore completa com n nós é $\lfloor \log_2 n \rfloor$, a inserção faz no máximo $\lfloor \log_2 n \rfloor$ trocas: custo O(log n) no pior caso. No melhor caso (o elemento já é menor que o pai) o custo é O(1).

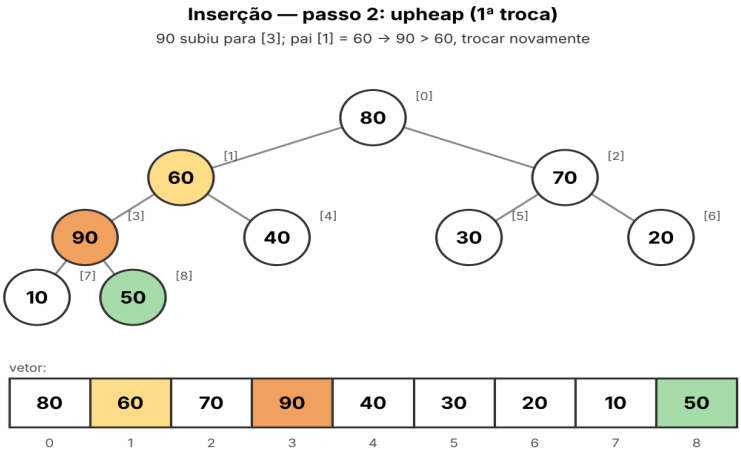
Exemplo passo a passo (upheap)

Considere o heap da Figura 1 e a inserção do valor 90. Nas figuras, a cor laranja indica o elemento que está subindo, a amarela o elemento com o qual ele é comparado e a verde o elemento que acabou de ser trocado.

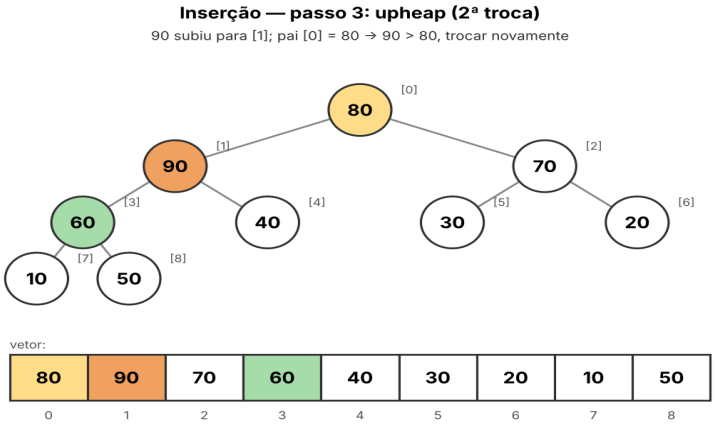




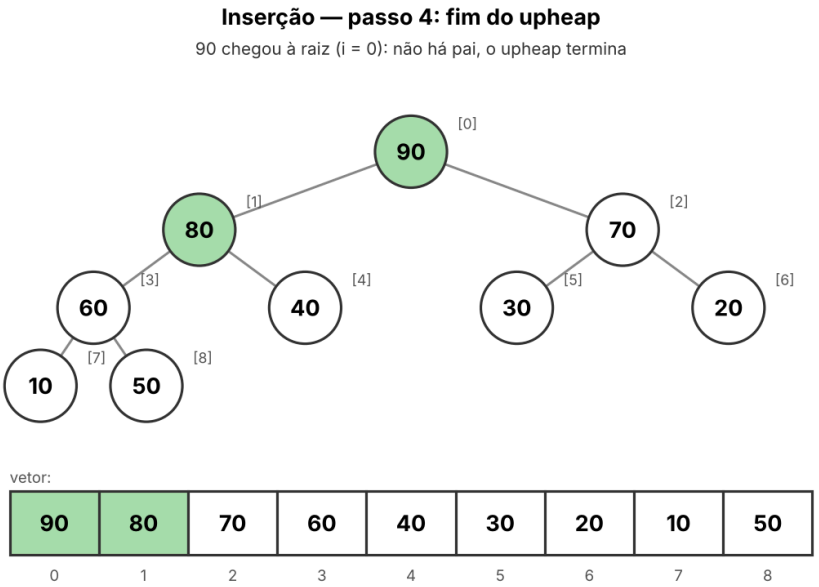
O valor 90 é colocado em dados[8]. Seu pai é dados[3] = 50; como 90 > 50, a propriedade de heap está violada.



Após a troca, 90 está em dados[3] e seu novo pai é dados[1] = 60. Como 90 > 60, troca-se novamente.



Agora 90 está em dados[1] e seu pai é dados[0] = 80. Como 90 > 80, ocorre a terceira troca.



- Remoção

Na lista de prioridades, a remoção retira o elemento de maior prioridade, que está na raiz. Retirar a raiz deixaria um buraco no início do vetor e quebraria a estrutura completa; por isso o algoritmo troca o papel da raiz com o último elemento do vetor. Esse último elemento é movido para a raiz e o tamanho do heap diminui em uma unidade, o que mantém a árvore completa. Contudo, o valor agora na raiz pode ser menor que o de seus filhos, e a propriedade de heap precisa ser restabelecida pelo **downheap**: o elemento é trocado com o maior de seus filhos enquanto for menor que ele.

Passos do algoritmo:

1. Se n = 0, o heap está vazio (erro).
2. Guardar dados[0] como valor a ser devolvido.
3. Copiar o último elemento, dados[n − 1], para dados[0] e decrementar n.
4. Executar downheap a partir do índice i = 0.
5. Devolver o valor guardado.

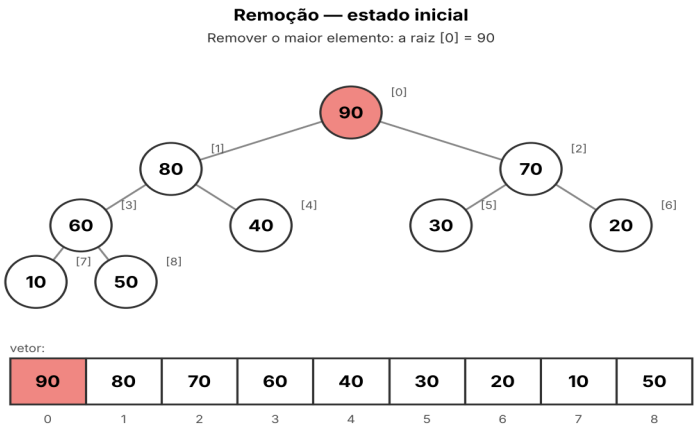
```
void downheap(Heap *h, int i) {
    for (;;) {
        int esq = 2 * i + 1, dir = 2 * i + 2, maior = i;
        if (esq < h->n && h->dados[esq] > h->dados[maior])
            maior = esq;
        if (dir < h->n && h->dados[dir] > h->dados[maior])
            maior = dir;
        if (maior == i)
            break;
        int tmp = h->dados[i];
        h->dados[i] = h->dados[maior];
        h->dados[maior] = tmp;
        i = maior;
    }
}
```

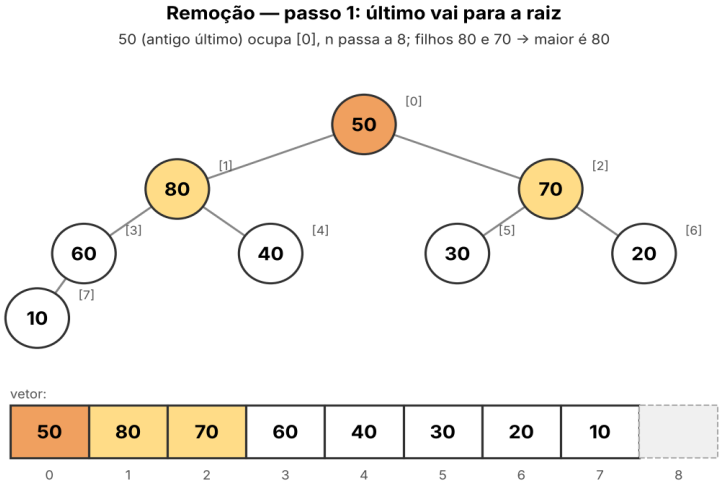
```c
int remover(Heap *h, int *saida) {
        if (h->n == 0) return 0;            /* heap vazio         */
        *saida = h->dados[0];               /* maior prioridade   */
        h->dados[0] = h->dados[h->n - 1];   /* último vai p/ raiz */
        h->n--;
        downheap(h, 0);
        return 1;
}
```

É essencial escolher o maior dos dois filhos: se o elemento fosse trocado com o menor, o filho maior ficaria abaixo de um pai menor que ele e a propriedade continuaria violada. Cada troca desce um nível, então o custo da remoção é O(log n) no pior caso.

Exemplo passo a passo (downheap)

Considere o heap resultante da inserção anterior e a remoção do maior elemento (90). Nas figuras, a cor vermelha indica o elemento removido, a laranja o elemento que está descendo, a amarela os filhos comparados e a verde o elemento que acabou de subir.

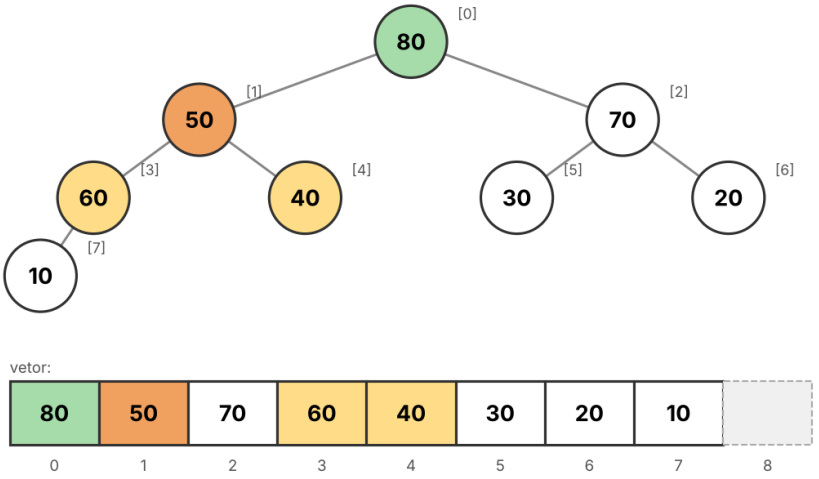




O valor 90 é guardado para ser devolvido. O último elemento (50, em dados[8]) é copiado para a raiz e n passa de 9 para 8. Os filhos da raiz são 80 e 70; o maior é 80 > 50, então haverá troca com o filho esquerdo.

Remoção — passo 2: downheap (1ª troca)

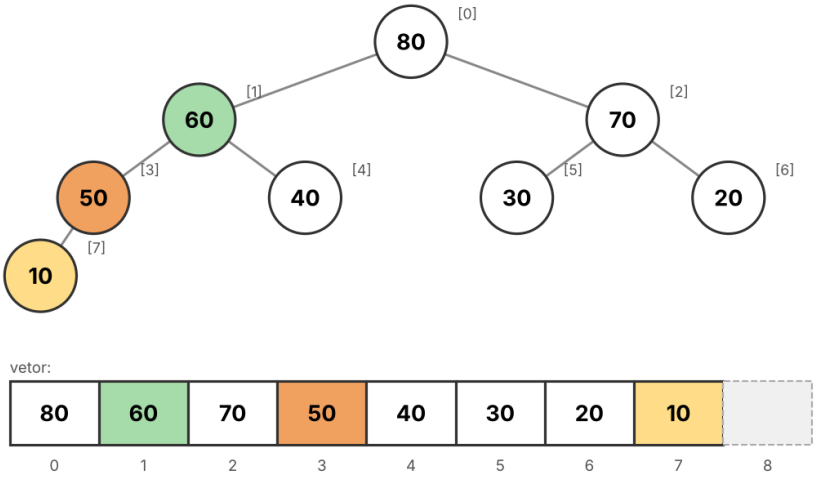
50 desceu para [1]; filhos 60 e 40 → maior é 60 > 50, trocar



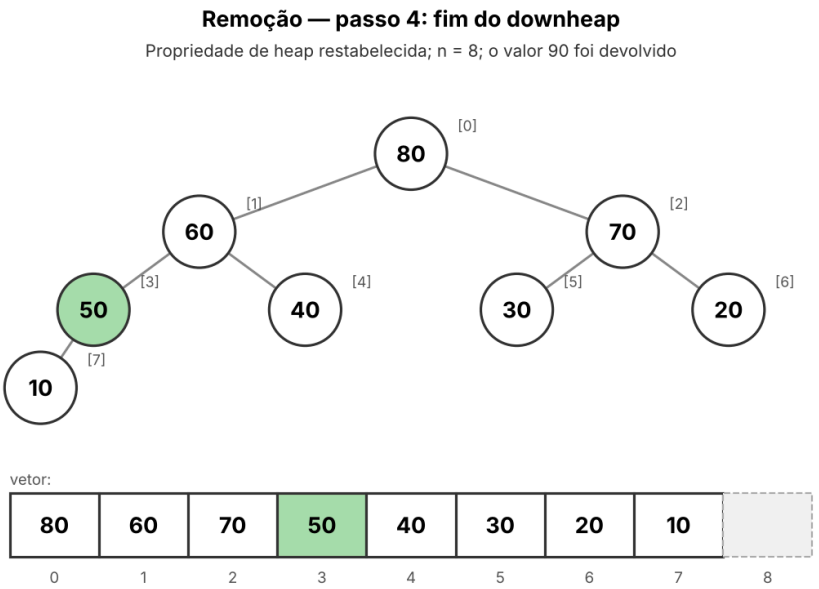
O valor 50 está em dados[1], cujos filhos são dados[3] = 60 e dados[4] = 40. O maior é 60 > 50, então ocorre a segunda troca.

Remoção — passo 3: downheap (2ª troca)

50 desceu para [3]; único filho [7] = 10 → 50 > 10, não há troca



O valor 50 está em dados[3]. Seu filho esquerdo seria dados[7] = 10; o filho direito (índice 8) já não pertence ao heap, pois n = 8. Como 50 > 10, não há troca.



O heap volta a satisfazer a propriedade, com n = 8, e a operação devolve o valor 90.

Complexidade

Operação	Pior caso	Observação
Consultar o maior	O(1)	dados[0]
Inserir	O(log n)	upheap: no máximo $\lfloor \log_2 n \rfloor$ trocas
Remover o maior	O(log n)	downheap: no máximo $\lfloor \log_2 n \rfloor$ trocas

Se o heap tem n elementos, o espaço utilizado é O(n), sem ponteiros adicionais. Em comparação, uma lista ordenada tem remoção em O(1) mas inserção em O(n), e uma lista não ordenada tem inserção em O(1) mas remoção do maior em O(n).

Fontes

Nota sobre as imagens

As figuras deste relatório (passo a passo da inserção e da remoção) foram geradas com o auxílio de inteligência artificial (Claude).

Referências

CORMEN, Thomas H. et al. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

DROZDEK, Adam. *Estrutura de dados e algoritmos em C++*. Tradução da 4. ed. norte-americana. [S. l.]: Cengage Learning Brasil, 2018.

FALCÃO, Alexandre Xavier. *Heap binário*. Campinas: UNICAMP, Instituto de Computação, MC202. Disponível em: https://www.ic.unicamp.br/~afalcao/mc202/HeapBinario.pdf. Acesso em: 6 nov. 2026.